In a general sense, you want Redis to have some sort of failover so that if the Redis you are using is lost, the system will automatically transition your app to an up-to-date backup. Many Redis vendors provide this, and it can be set up in a self– or cloud-hosted environment. Even still, there is a chance a job could be lost.

While you can't easily monitor for lost jobs directly, you really don't care about them unless that leads to a negative business outcome. This is why monitoring business functions is so critical. In the example app, you would want to monitor all orders created but not charged, or whose email notification hasn't gone out, or where fulfillment has not been requested. If that happens, you can correlate all your measures to figure out why, including a Redis failover happening during the time the order was created.

This is a deep topic, but what to take away here is that Redis is not an ACID-compliant database,[21] and the only way to be sure your business functions complete is to *monitor* them.

Alert on Actionable Problems

If you've followed the advice above, you will have a *lot* of measurements flowing into your monitoring system. Almost none of these can be used to identify problems on their own. For example, paging the entire engineering team when the Redis server's CPU usage goes above 90 percent isn't very useful, because it's not clear what actual problem is happening.

This includes the various statistics about Sidekiq or your specific jobs. You need to be able to view the performance over time, but you should *alert* on business-level outcomes that are not normal, for example, the rate of order creation and completion. Observe those outcomes over time to establish a baseline, then set up alerting.

That being said, you should make sure to alert when Redis is running out of free space. Since memory is finite, when Redis' memory fills up, your entire